AutoChip Lab Report

Overview

Seven of the eight tutorials passed their testbenches: five in the first iteration of the first run, and two after the prompt was revised. The 8-bit CPU passed its testbench in the first AutoChip iteration, after a three-turn architecture conversation. All designs were generated with gpt-4o-mini, using 5 candidates per iteration and up to 6 iterations, and verified with Icarus Verilog (iverilog -g2012, then vvp). Each testbench compared the design against a reference model and printed "Mismatches: X in Y samples", which AutoChip used both to rank candidates and as feedback. Claude was consulted to create the prompts and testbenches. Most failures came from a mistake the model repeated in every candidate, which the mismatch feedback did not explain.

Part I:

Example 1 - binary_to_bcd

Initial prompt: Combinational converter from a 5-bit binary input to an 8-bit BCD output (tens in [7:4], ones in [3:0]).

Problems: Iteration 0 had a compile error and a warning; feedback removed both by iteration 1. Every candidate then got the same 44 of 64 samples wrong (inputs 10 to 31) through iteration 5. The model checked the ones digit for 10 or more after shifting, instead of 5 or more before shifting.

Revision: The prompt added the double-dabble order: add 3 to any digit of 5 or more, then shift.

Final status: Successful. Passed in iteration 0, with 4 of 5 candidates at 0 mismatches.

Example 2 - sequence_detector

Initial prompt: Detect an 8-value sequence of 3-bit inputs sampled while enabled, with overlapping matches and an asynchronous reset.

Problems: The best candidate improved from 330 to 161 mismatches out of 3019 but never passed. It used a circular buffer that only checked every 8th sample and compared a slot that was being overwritten on the same edge.

Revision: A hint asked for a 24-bit shift register compared as {history[20:0], data}. Every candidate switched to a shift register but stayed at 177 mismatches, equal to the number of detections. The target constant was written as 18 garbled bits, so the output never went high.

Final status: Incomplete. The next revision would give the exact 24-bit constant in the prompt, or use a stronger model for later iterations.

Example 3 - shift_register

Initial prompt: 8-bit left-shift register with serial data input, shift enable, and active-low asynchronous reset.

Result: All 5 candidates passed in iteration 0 (0 mismatches in 1017 samples).

Final status: Successful.

Example 4 - abro_state_machine

Initial prompt: Wait until A and B have both been seen, in either order, then hold O = 1 until reset, with one-hot states output on State.

Result: 4 of 5 candidates passed in iteration 0 (0 mismatches in 2243 samples); the fifth had 287.

Final status: Successful.

Example 5 - dice_roller

Initial prompt: Roll a 4, 6, 8, or 20-sided die on a rising edge of roll, using a 16-bit LFSR and (lfsr % sides) + 1.

Result: All 5 candidates passed in iteration 0 (0 mismatches in 3011 samples, 766 rolls). Unlike the ChipChat testbench, which only checked the range, this one checked each roll's exact value.

Final status: Successful.

Example 6 - lfsr

Initial prompt: 8-bit LFSR, initial state 10001010, taps at positions 1, 4, 6, 7, given as data[0], data[3], data[5], data[6].

Result: All 5 candidates passed in iteration 0 (0 mismatches in 604 samples).

Final status: Successful.

Example 7 - sequence_generator

Initial prompt: Output 8 hex values in order and repeat while enabled, with an asynchronous reset to the first value.

Problems: All 30 candidates failed to compile because the model named its value array sequence, which is a SystemVerilog keyword under -g2012. iverilog reported only "syntax error", so the feedback could not fix it. A second prompt warning against the keyword failed the same way in all 30 candidates.

Revision: The third prompt specified a 3-bit index register and the full case table for data.

Final status: Successful. All 5 candidates passed in iteration 0 (0 mismatches in 1019 samples).

Example 8 - traffic_light

Initial prompt: Red for 32 enabled cycles, green for 20, yellow for 7, with enable and an asynchronous reset.

Result: Only 1 of 5 candidates passed in iteration 0 (0 mismatches in 3005 samples). Three had about 1290 mismatches and one did not compile. AutoChip's ranking selected the correct one.

Final status: Successful.

Part II: 8-bit CPU

The architecture was developed over three conversational turns with gpt-4o-mini, with the full message history kept each turn. The RTL was then generated with AutoChip. Claude was consulted for the prompts.

1. **Starting prompt:** The co-design prompt from the assignment. The model returned a generic plan with no concrete design. It suggested up to 256 memory locations, dual-port memory, FSM control, and designing its own standard cells.
2. **Concrete ISA:** I corrected these points and asked for an opcode/operand split, memory size, and full instruction table. The model chose a 4-bit opcode and 4-bit operand with 16 bytes of memory. However, it said only 15 addresses were usable, stored a constant over the first instruction, left 6 opcodes reserved, and set flags without a flag register.
3. **ISA freeze:** I listed each flaw and froze the ISA: 16 opcodes including LDI, JC, SHL, and SHR, HALT = 1101, a carry flag, and single-cycle execution. The model approved it without finding a real issue. Its cell estimate gave 96 cells for 128 flip-flops, and its example program encoded JZ 3 as 0x80 and stored data over its own instructions.
4. **Testbench:** An instruction-level reference model ran 3 hand-written programs and 30 random programs. It checked ACC, PC, carry, and halted every cycle, and all 16 memory words at the end (2418 samples).
5. **AutoChip RTL:** The frozen ISA was given as a single prompt, with memory declared as reg [7:0] mem [0:15] so the testbench could load programs. In iteration 0, one candidate passed; the others had 25, 168, and 1569 mismatches, and one did not compile. AutoChip selected the passing design, so no feedback iterations were needed.

Final design: Single-cycle 8-bit accumulator CPU with 16 one-byte instructions, a 4-bit PC, a carry flag, a halted flag, and 16 bytes of memory.

Final status: Successful. 0 mismatches in 2418 samples across all 33 programs.

Limitations:

- **Scope:** 16 bytes of memory, no I/O or subroutines, and no synthesis check against the 1000-cell budget.
- **Architecture:** AutoChip needs a fixed testbench, so it could not design the architecture. That was done in conversation, and every major decision was made or corrected by hand.
- **Interface:** A carry_out port was added so the testbench could check the carry flag.
- **Feedback loop:** Because the CPU passed in iteration 0, the feedback loop was never tested on it.

Example of Feedback That Changed the RTL

In sequence_detector's first run, the best design from each iteration was simulated and its mismatch lines were fed back. In iteration 0, two candidates did not compile and the best had 330 mismatches. After feedback, the best improved to 213 mismatches in iteration 1 and 161 in iteration 3. The feedback

showed which cycles failed and the expected last 8 values, but not why. The final candidate still used a circular buffer, so it never passed.

Conclusion

Designs passed in the first iteration when the prompt stated the exact algorithm, update rule, or structure. Feedback fixed compile errors but rarely fixed logic errors. When every candidate made the same mistake, as in binary_to_bcd, sequence_detector, and sequence_generator, the mismatch counts did not lead the model to the cause, and a prompt revision was needed. Candidate ranking still helped: in traffic_light and the CPU, only one of five candidates was correct, and AutoChip picked it.